Copilot Instructions — Madagascar Freshwater Fish Conservation Platform

Project Purpose

Centralized open-source platform for conservation coordination of Madagascar's ~79 endemic freshwater fish species. Integrates public species profiles, ex-situ captive population tracking, in-situ field program monitoring, and cross-sector networking (zoos, researchers, hobbyist breeders). Designed to complement — not replace — ZIMS, IUCN Red List, FishBase, and GBIF.

Tech Stack

Layer	Technology
Backend	Python 3.12+, Django 5.x, Django REST Framework
Database	PostgreSQL 16 + PostGIS 3.4
Frontend	Next.js 14 (React, TypeScript)
Task Queue	Celery + Redis
Cache / Sessions	Redis
File Storage	MinIO (dev), S3-compatible (production)
Linting	Ruff (Python), ESLint + tsc (TypeScript)
Testing	pytest (Django), Vitest (Next.js)
Containerization	Docker Compose

Build, Test, and Lint Commands

> *Commands to be filled in after initial scaffolding. See `CLAUDE.md` → "Development Commands".*

Planned toolchain:

- `ruff check .` — Python lint
- `ruff format .` — Python format
- `pytest` — full Django test suite
- `pytest path/to/test_file.py::TestClass::test_method` — single test
- `python manage.py migrate` — apply migrations
- `python manage.py runserver` — dev server
- `npx tsc --noEmit` — TypeScript type check
- `vitest run` — frontend tests

Architecture Overview

The system uses a **decoupled layered architecture**:

```
Next.js (SSR + SPA)  ←→  DRF REST API  ←→  Django App Core  ←→  PostgreSQL/PostGIS
                                                  ↕
                                            Celery Workers (Redis)
                                            - IUCN/FishBase sync
                                            - Darwin Core Archive generation
                                            - GBIF publishing
```

- The Next.js frontend communicates **exclusively** via the DRF API — no direct database access.
- Django Admin is used for early-phase data entry before the full frontend is complete.
- All external API calls (IUCN, FishBase, GBIF) are handled by **Celery tasks**, not in request cycles. Results are cached in Redis and persisted to PostgreSQL.
- Django signals trigger downstream updates when species assessments or population records change (recalculate prioritization scores, flag GBIF re-publish, invalidate cache).

Django Application Structure

App	Responsibility
`species`	Taxonomy, species profiles, IUCN status, FishBase data, common names, distributions
`populations`	Ex-situ population tracking: institutions, holding records, breeding events, transfers, censuses
`fieldwork`	In-situ field programs, surveys, occurrence records (Darwin Core), eDNA samples
`coordination`	Breeding recommendations, prioritization scores, action plans
`accounts`	Users, institutional affiliations, access tiers, audit log
`integration`	External API sync clients, Darwin Core serialization, GBIF publishing, import pipelines

Five-Tier Access Model

Access tier is stored as an integer (1–5) on the `User` model and enforced at three levels:

1. **DRF serializers** — use `SerializerMethodField` to conditionally include/exclude fields based on `request.user.access_tier`
2. **Model managers** — provide `.for_tier(tier)` querysets that filter sensitive records
3. **DRF permission class** — `TierPermission(min_tier=N)` gates endpoint access

Tier	Role	Key Data Access
1	Public (anonymous)	Species profiles, conservation status, aggregated ex-situ counts
2	Registered Researcher	Occurrence datasets, field program details, generalized (0.1°) locations
3	Conservation Coordinator	Exact locations, per-institution population detail, breeding recommendations, transfers
4	Program Manager	Genetic data, pedigree, studbook-level records
5	Administrator	Full access, user management, audit logs

Institution-scoping: Tier 3–4 users can only edit records for their affiliated institution(s). Enforced at queryset level, not just view level.

Coordinate Generalization (Critical)

Occurrence coordinates for threatened species are **stored exact in PostGIS**, then generalized at the API serializer level:

Tier	Precision Returned
1 (Public)	Country + broad region text only
2 (Researcher)	0.1 degree (~11 km)
3+ (Coordinator)	Exact coordinates

Never expose exact coordinates for threatened species to Tier 1–2 users. Generalization logic belongs in the serializer layer, not in views or models.

Data Model Highlights

- **`OccurrenceRecord`** — Darwin Core–compliant. Stores full DwC fields in a JSONB column (`darwin_core_fields`) plus indexed copies of high-frequency fields (`decimalLatitude` , `decimalLongitude` , `eventDate` , `basisOfRecord` , `scientificName`). Dual geometry columns: `location` (exact PostGIS Point) and `location_generalized` (public-safe Point).
- **`ExSituPopulation`** — Central tracking unit for ex-situ coordination. Aggregated counts are public; per-institution detail is Tier 3+; genetics are Tier 4+.
- **`Species`** — Holds `external_ids` JSONB with `iucn_taxon_id` , `fishbase_id` , `gbif_taxon_key` for cross-system linking.
- **`AuditLog`** — Immutable append-only table. Never update or delete rows.
- **`Taxon`** — Hierarchical taxonomy using MPTT (nested sets). Represents family → genus → species.

Darwin Core Compliance

- Occurrence data follows Darwin Core standard for GBIF publishing via Darwin Core Archive (DwC-A) files.
- DwC-A generation runs as a Celery task; output ZIP files stored in S3.
- Custom extensions are used for breeding coordination data that doesn't fit DwC. Maintain DwC compliance **only** for the GBIF-publishing subset (occurrence/species data).
- CI validates DwC-A output using custom pytest fixtures against GBIF validator on every PR touching the `integration` app.

External Integrations

System	Direction	Method
IUCN Red List API	Read	REST (token-based), weekly Celery sync, 7-day Redis cache
FishBase	Read	rfishbase/REST, weekly Celery sync
GBIF	Write	Darwin Core Archive + IPT, on-demand Celery task
ZIMS/Species360	Read	Manual CSV import pipeline (no public API)

CARES Priority Lists	Read	Manual import (static HTML scrape)

Secrets & Environment

- All secrets (Django `SECRET_KEY`, IUCN/GBIF API tokens, DB credentials) are stored in `.env` files.
- `.env` files are excluded by `.gitignore`. Never commit secrets.
- Environment variables are the only acceptable mechanism for credentials.

Agent Delegation Workflow

This project uses specialized Claude agents in `.claude/agents/`. The standard workflow is:

Architecture → BA → PM → [implement + unit tests] → Test Writer + Reviewers

- `@architecture` — technical design decisions before implementation
- `@business-analyst` — evaluates WHAT to build; acceptance criteria
- `@product-manager` — plans HOW to build it; stories and gates
- `@security-reviewer` — invoke on any change touching auth, permissions, data access, or API endpoints
- `@test-writer` — writes tests from acceptance criteria (not implementation) at gate checkpoints
- `@code-quality-reviewer` — maintainability and pattern adherence reviews

Planning documents live in `docs/planning/` (architecture, business-analysis, specs subdirs).
Ideation and domain context live in `docs/ideation/`.